

基于算法蒸馏的上下文强化学习

迈克尔·拉斯金* 王璐宇* 吴俊赫 埃米利奥·帕里索托 斯蒂芬·斯宾塞

里奇·斯泰格瓦尔德 DJ·斯特劳斯 史蒂文·汉森 安杰洛斯·菲洛斯 伊桑·布鲁克斯

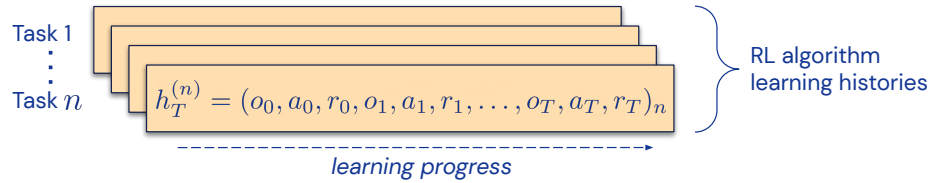
马克西姆·加佐 希曼舒·萨尼 萨廷德·辛格 沃洛季米尔·姆尼赫

深度思维公司

摘要

我们提出算法蒸馏（Algorithm Distillation, AD），一种通过用因果序列模型建模强化学习（RL）算法的训练历史，将强化学习算法蒸馏到神经网络中的方法。算法蒸馏将学习强化学习视为一个跨回合的顺序预测问题。学习历史数据集由源强化学习算法生成，然后通过自回归地预测动作（给定其之前的学习历史作为上下文）来训练因果变换器。与蒸馏学习后或专家序列的顺序策略预测架构不同，AD 能够完全在上下文中改进其策略，而无需更新其网络参数。我们证明 AD 可以在具有稀疏奖励、组合任务结构和基于像素的观察的各种环境中进行上下文强化学习，并发现 AD 学习到的强化学习算法比生成源数据的算法更具数据效率。

Data Generation



Model Training

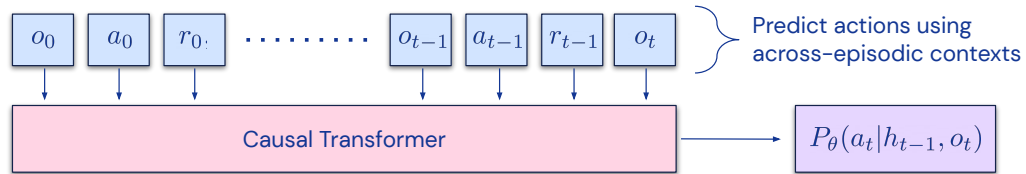


图 1：算法蒸馏（AD）有两个步骤——（i）从解决不同任务的单个单任务强化学习算法中收集学习历史数据集；（ii）因果变换器使用跨回合上下文从这些历史中预测动作。由于强化学习策略在整个学习历史中不断改进，通过准确预测动作，AD 学会输出相对于其上下文中看到的策略有所改进的策略。AD 建模状态-动作-奖励标记，并且不以回报为条件。

* 同等贡献。通讯作者：mlaskin, luyuwang, vminh@deepmind.com。

1 引言

变换器已成为序列建模的强大神经网络架构 (Vaswani et al., 2017)。预训练变换器的一个显著特性是它们能够通过提示条件化或上下文学习来适应下游任务。在大型离线数据集上预训练后，大型变换器已被证明可以泛化到文本补全 (Brown et al., 2020)、语言理解 (Devlin et al., 2018) 和图像生成 (Yu et al., 2022) 等下游任务。

近期研究表明，变换器 (Transformer) 也能通过将离线强化学习 (Reinforcement Learning, RL) 视为序列预测问题，从离线数据中学习策略。Chen 等人 (2021) 证明变换器可通过模仿学习从离线强化学习数据中学习单任务策略，后续工作进一步表明变换器能在同域 (Lee 等人, 2022) 和跨域 (Reed 等人, 2022) 设置中提取多任务策略。这些工作揭示了一种有前景的范式，用于提取通用多任务策略——首先收集大规模且多样化的环境交互数据集，然后通过序列建模从数据中提取策略。我们将这类通过模仿学习从离线强化学习数据中学习策略的方法统称为离线策略蒸馏 (Offline Policy Distillation)，或简称为策略蒸馏 (Policy Distillation)¹ (PD)。

尽管策略蒸馏简单且可扩展性强，但其一个显著缺点是所得策略无法通过与环境的额外交互逐步改进。例如，多游戏决策变换器 (Multi-Game Decision Transformer, MGD, Lee 等人, 2022) 学习了一个以回报为条件的策略，能够玩多种 Atari 游戏；而 Gato (Reed 等人, 2022) 则通过上下文推断任务，学习了一个跨多种环境解决任务的策略。然而，这两种方法都无法通过试错在上下文中改进其策略。MGD 通过微调模型权重来适应新任务，而 Gato 则需要专家演示提示才能适应新任务。简言之，策略蒸馏方法学习的是策略，而非强化学习算法。

我们假设策略蒸馏无法通过试错改进的原因在于，其训练数据未展示学习过程。当前方法要么从无学习过程的数据中学习策略（例如蒸馏固定的专家策略），要么从有学习过程的数据（例如强化学习智能体的经验回放缓冲区）中学习，但上下文长度过小，无法捕捉策略改进。

我们的核心观察是，强化学习算法训练中学习的顺序性原则上使得将强化学习过程本身建模为因果序列预测问题成为可能。具体而言，如果变换器的上下文足够长，能够包含由学习更新带来的策略改进，那么它应当不仅能表示固定策略，还能通过关注先前回合中的状态、动作和奖励来表示策略改进算子。这开启了将任何强化学习算法通过模仿学习蒸馏为足够强大的序列模型（如变换器）的可能性，从而将其转化为上下文强化学习算法。

我们提出算法蒸馏 (Algorithm Distillation, AD)，一种通过优化强化学习算法学习历史之上的因果序列预测损失来学习上下文策略改进算子的方法。AD 包含两个组成部分。首先，通过保存强化学习算法在多个独立任务上的训练历史，生成一个大规模多任务数据集。接着，变换器以先前的学习历史作为上下文，对动作进行因果建模。由于源强化学习算法在训练过程中策略不断改进，AD 被迫学习改进算子，以便准确建模训练历史中任意给定点的动作。关键在于，变换器的上下文大小必须足够大（即跨回合），以捕捉训练数据中的改进。完整方法如图 1 所示。

我们证明，通过使用具有足够大上下文的因果变换器模仿基于梯度的强化学习算法，AD 能够完全在上下文中强化学习新任务。我们在多个需要探索的部分可观测环境中评估 AD，包括来自 DMLab (Beattie et al., 2016) 的基于像素的水迷宫 (Morris, 1981)。我们证明 AD 具备上下文探索、时间信用分配和泛化能力。我们还证明 AD 学习到的算法比生成变换器训练源数据的算法更具数据效率。据我们所知，AD 是首个通过使用模仿损失对离线数据进行序列建模来展示上下文强化学习的方法。

¹ 我们所说的策略蒸馏与 Rusu et al. (2016) 类似，但策略是从离线数据中蒸馏的，而非教师网络。

2 背景

部分可观测马尔可夫决策过程：马尔可夫决策过程（Markov Decision Process, MDP）由状态 $s \in \mathcal{S}$ 、动作 $a \in \mathcal{A}$ 、奖励 $r \in \mathcal{R}$ 、折扣因子 γ 和转移概率函数 $p(s_{t+1}|s_t, a_t)$ 组成，其中 t 是表示时间步的整数， $(\mathcal{S}, \mathcal{A})$ 是状态空间和动作空间。在由 MDP 描述的环境中，每个时间步 t 智能体观测状态 s_t ，从其策略中选择动作 $a_t \sim \pi(\cdot|s_t)$ ，然后观测从环境转移动态中采样的下一状态 $s_{t+1} \sim p(\cdot|s_t, a_t)$ 。本文工作在部分可观测马尔可夫决策过程（Partially Observable Markov Decision Process, POMDP）设定下进行，其中智能体接收观测 $o \in \mathcal{O}$ 而非状态 $s \in \mathcal{S}$ ，这些观测仅包含环境真实状态的部分信息。

完整的状态信息可能不完整，原因在于环境中缺少关于目标的信息，智能体必须通过带记忆的奖励来推断，或者因为观测是基于像素的，或者两者兼有。

在线与离线强化学习：强化学习算法旨在最大化回报，回报定义为智能体整个生命周期或训练回合中奖励的累积和 $\sum_t \gamma^t r_t$ 。强化学习算法大致分为两类：同策略算法（Williams, 1992），智能体直接最大化总回报的蒙特卡洛估计；异策略算法（Mnih et al., 2013; 2015），智能体学习并最大化近似未来总回报的价值函数。大多数强化学习算法通过直接与环境交互的试错来最大化回报。然而，离线强化学习（Levine et al., 2020）最近作为一种替代且通常互补的强化学习范式出现，智能体旨在从另一智能体收集的离线数据中提取回报最大化策略。离线数据集由 (s, a, r) 元组组成，通常用于训练异策略智能体，尽管从离线数据中提取回报最大化策略的其他算法也是可能的。

自注意力与变换器 自注意力（Vaswani et al., 2017）操作首先用三个独立的矩阵将输入数据 X 投影到 D 维向量，分别称为查询 Q 、键 K 和值 V 。然后这些向量通过注意力函数：

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{D})V. \quad (1)$$

QK^T 项计算输入数据 X 的两个投影之间的内积。该内积随后被归一化，并通过缩放项 V 投影回 D 维向量。变换器（Transformer）（Vaswani 等人，2017；Devlin 等人，2018；Brown 等人，2020）利用自注意力作为架构的核心部分来处理序列数据，如文本序列。变换器通常通过自监督目标进行预训练，该目标预测序列数据中的标记。常见的预测任务包括预测随机掩码的标记（Devlin 等人，2018）或应用因果掩码并预测下一个标记（Radford 等人，2018）。

离线策略蒸馏：我们将将离线强化学习视为序列预测问题的方法族称为离线策略蒸馏，或简称为策略蒸馏

（PD）。PD 不是从离线数据中学习价值函数，而是通过使用序列模型预测离线数据中的动作（即行为克隆）以及回报条件（Chen 等人，2021；Lee 等人，2022）或过滤掉次优数据（Reed 等人，2022）来提取策略。

最初提出用于学习单任务策略（Chen 等人，2021；Janner 等人，2021），PD 最近被扩展为从多样化的离线数据中学习多任务策略（Lee 等人，2022；Reed 等人，2022）。

上下文学习：上下文学习指的是从上下文中推断任务的能力。例如，像 GPT-3（Brown 等人，2020）或 Gopher（Rae 等人，2021）这样的大语言模型可以通过将任务以语言形式指定为提示来执行文本补全、代码生成和文本摘要等任务。这种从提示中推断任务的能力通常被称为上下文学习。我们使用先前序列模型工作中的术语“权重内学习”和“上下文学习”（Brown 等人，2020；Chan 等人，2022）来分别区分基于梯度的参数更新学习和基于上下文的无梯度学习。

3 方法

在一个有能力的强化学习（RL）智能体的整个生命周期中，它会表现出复杂的行为，如探索、时间信用分配和规划。我们的核心洞见是，一个

智能体的动作，无论其所处环境、内部结构和实现方式如何，都可以视为其过去经验的函数，我们称之为历史。形式上，我们写为：

$$\mathcal{H} \ni h_t := (o_0, a_0, r_0, \dots, o_{t-1}, a_{t-1}, r_{t-1}, o_t, a_t, r_t) = (o_{\leq t}, r_{\leq t}, a_{\leq t}) \quad (2)$$

我们将一个长的 2 以历史为条件的策略称为算法：

$$P : \mathcal{H} \cup \mathcal{O} \rightarrow \Delta(\mathcal{A}), \quad (3)$$

其中 $\Delta(\mathcal{A})$ 表示动作空间 $\mathcal{A}$ 上的概率分布空间。公式 (3) 表明，与策略类似，算法可以在环境中展开以生成观察、奖励和动作的序列。为简洁起见，我们将算法记为 F ，环境（即任务）记为 $\mathcal{M}$ ，使得任何给定任务 $\mathcal{M}$ 的学习历史由算法 $P_{\mathcal{M}}$ 生成。

$$(O_0, A_0, R_0, \dots, O_T, A_T, R_T) \sim P_{\mathcal{M}}. \quad (4)$$

这里，我们用大写拉丁字母 *e.g.* O, A, R 表示随机变量，用小写拉丁字母 *e.g.* o, a, r 表示它们的值。通过将算法视为以长历史为条件的策略，我们假设任何生成一组学习历史的算法都可以通过对动作进行行为克隆来蒸馏到神经网络中。接下来，我们提出一种方法，在给定智能体生命周期的情况下，通过行为克隆学习一个序列模型，将长历史映射到动作分布。

3.1 算法蒸馏

假设智能体的生命周期，我们也称之为学习历史，是由源算法 P^{source} 针对许多单独任务 $\{\mathcal{M}_n\}_{n=1}^N$ 生成的，从而产生数据集 $\mathcal{D}$ ：

$$\mathcal{D} := \left\{ \left(o_0^{(n)}, a_0^{(n)}, r_0^{(n)}, \dots, o_T^{(n)}, a_T^{(n)}, r_T^{(n)} \right) \sim P_{\mathcal{M}_n}^{\text{source}} \right\}_{n=1}^N. \quad (5)$$

然后，我们通过负对数似然 (NLL) 损失将源算法的行为蒸馏到一个序列模型中，该模型将长历史映射到动作概率，并将此过程称为算法蒸馏 (AD)。在这项工作中，我们考虑具有参数 θ 的神经网络模型 P_θ ，通过最小化以下损失函数来训练它们：

$$\mathcal{L}(\theta) := - \sum_{n=1}^N \sum_{t=1}^{T-1} \log P_\theta(A = a_t^{(n)} | h_{t-1}^{(n)}, o_t^{(n)}). \quad (6)$$

直观地说，一个具有固定参数、通过算法蒸馏 (AD) 训练的序列模型应当摊销源强化学习算法 P^{source} ，并因此表现出类似复杂的行为，如探索和时间信用分配。由于强化学习策略在源算法的学习历史中不断改进，准确的动作预测要求序列模型不仅从前文上下文中推断当前策略，还要推断改进后的策略，从而蒸馏出策略改进算子。

3.2 实用实现

在实践中，我们将算法蒸馏实现为一个两步过程。首先，通过在许多不同任务上运行单个基于梯度的强化学习算法来收集学习历史数据集。接下来，训练一个具有多回合上下文的序列模型，从历史中预测动作。我们在下面描述这两个步骤，并在算法 1 中详细说明完整的实用实现。

数据集生成： 通过训练 N 个单任务基于梯度的强化学习算法来收集学习历史数据集。为了防止序列模型训练期间对任何特定任务过拟合，每次强化学习运行都从任务分布中随机采样一个任务 $\mathcal{M}$ 。数据生成步骤与强化学习算法无关——任何强化学习算法都可以被蒸馏。我们展示了蒸馏 UCB 探索 (Lai & Robbins, 1985)、在线演员-评论家 (Mnih et al., 2016) 和离线 DQN (Mnih et al., 2013) 的结果，包括分布式和单流设置。我们将学习历史数据集表示为公式 5 中的 $\mathcal{D}$ 。训练序列模型：一旦收集到学习历史数据集 $\mathcal{D}$ ，就训练一个序列预测模型，根据前面的历史来预测动作。 2 足够长以跨越学习更新，例如跨回合。

Algorithm 1 Algorithm Distillation

Require: Train $\{\mathcal{M}^{\text{train}}\}$ and test $\{\mathcal{M}^{\text{test}}\}$ tasks, observations $o \in \mathcal{O}$, actions $a \in \mathcal{A}$, and rewards $r \in \mathcal{R}$.
Require: Network parameters ϕ_i for $i = 1, \dots, N$ source RL algorithms.
Require: Network parameters θ for a causal transformer P_θ that predicts actions.
Require: An empty buffer to store data $\mathcal{D}$.

- 1: **for** $i = 1 \dots N$ **do** ▷ Part 1: Dataset Generation
- 2: Sample a task $\mathcal{M}_i^{\text{train}}$ randomly from the train task distribution.
- 3: Train the source RL algorithm ϕ_i until it converges to the optimal policy.
- 4: Save the learning history $h_T^{(i)} = (o_0, a_0, r_0, \dots, o_T, a_T, r_T)_i$ to the dataset $\mathcal{D} \leftarrow \mathcal{D} \cup h_T^{(i)}$.
- 5: **end for**
- 6: **while** P_θ not converged **do** ▷ Part 2: Algorithm Distillation
- 7: Randomly sample a multi-episodic subsequence $\bar{h}_j^{(i)} = (o_j, a_j, r_j, \dots, o_{j+c}, a_{j+c}, r_{j+c})_i$ of length c .
- 8: Autoregressively predict the actions with P_θ and compute the NLL loss in Eq. 6.
- 9: Backpropagate to update the transformer parameters.
- 10: **end while**
- 11: **for** $k = 1 \dots M_{\text{seeds}}$ **do** ▷ Part 3: Autoregressive Evaluation
- 12: Sample a task $\mathcal{M}_k^{\text{test}}$ randomly from the test task distribution. Initialize empty context queue C .
- 13: Unroll the transformer $P_\theta(\cdot|C)$ in the environment storing sequential transitions (*i.e.* histories) in C .
- 14: Measure the return accumulated by the agent for each episode of evaluation.
- 15: **end for**

历史。我们利用 GPT (Radford et al., 2018) 因果变换器模型进行序列动作预测，但算法蒸馏与任何序列模型兼容，包括 RNN (Williams & Zipser, 1989)。例如，我们在附录 K 中展示，算法蒸馏也可以通过 LSTM (Hochreiter & Schmidhuber, 1997) 实现，尽管不如使用因果变换器的算法蒸馏有效。由于因果变换器的训练和推理在序列长度上是二次的，我们从 $\mathcal{D}$ 中采样长度为 $c < T$ 的跨回合子序列 $\bar{h}_j = (o_j, r_j, a_j, \dots, o_{j+c}, r_{j+c}, a_{j+c})$ ，而不是训练完整历史。

4 实验设置

4.1 环境

为研究 AD 及基线（见下一节）的上下文强化学习能力，我们聚焦于预训练后无法通过零样本泛化解决的环境。具体而言，要求每个环境支持众多任务，任务难以从观测中轻易推断，且回合长度足够短，以便可行地训练跨回合因果变换器——环境详情见附录 A。评估环境如下：

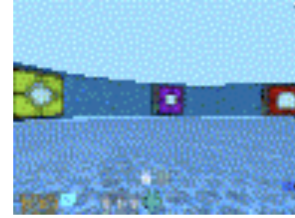


图 2: 来自 DM-Lab 水迷宫环境的智能体视角。任务是找到一个隐藏平台，一旦找到便会升起。

对抗性赌博机: 一个具有 10 个臂和 100 次试验的多臂赌博机，与 RL² (Duan 等人 (2016)) 考虑的环境类似。然而，评估期间奖励分布超出训练分布。训练期间，奖励在奇数臂下出现的概率为 95%。评估时情况相反，奖励在偶数臂下出现的概率为 95%。

暗室: 一个二维离散部分可观测马尔可夫决策过程，智能体在房间中生成，必须找到目标位置。智能体仅知道自身 (x, y) 坐标，但不知道目标位置，必须从奖励中推断。房间大小为 9×9 ，可选动作作为左移一步、右移一步、上移一步、下移一步及无操作，回合长度为 20，智能体在地图中心重置。我们测试两种环境变体——暗室（智能体每次到达目标时获得 $r = 1$ ）和困难暗室（一种困难探索变体，尺寸为 17×17 ，奖励稀疏（到达目标仅获得一次 $r = 1$ ））。未 $r = 1$ 时，则 $r = 0$ 。

暗钥匙开门: 与暗室类似，但该环境要求智能体首先找到一把不可见的钥匙，找到后获得一次 $r = 1$ 奖励，然后打开一扇不可见的门。

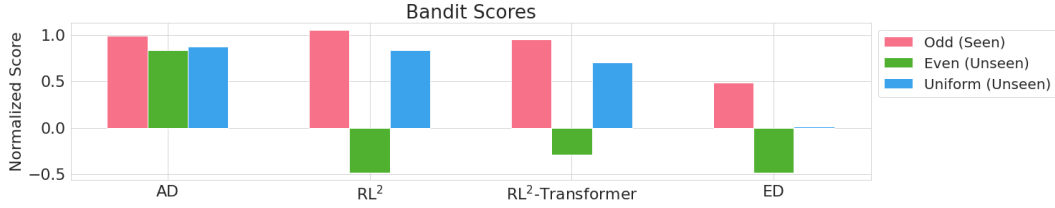


图 3: 对抗性赌博机 (Adversarial Bandit) (第 5): AD, RL² 节), 以及在 100 次试验的 10 臂赌博机上评估的 ED。AD 的源数据来自 UCB 的学习历史 (Lai & Robbins, 1985)。训练期间, 奖励在奇数臂下以 95% 的概率分布, 在评估期间在偶数臂下以 95% 的概率分布。AD 和 RL² 都能在上下文中学习分布内任务, 但 AD 在分布外泛化得更好。使用变换器运行 RL² 通常不会比原始 LSTM 变体提供优势。相对于 AD 和 RL², ED 在分布内和分布外都表现不佳。分数相对于 UCB 进行了归一化。

它会再次获得 $r = 1$ 的奖励。否则, 奖励为 $r = 0$ 。房间大小为 9×9 , 使得任务空间具有组合性, 共有 $81^2 = 6561$ 种可能任务。该环境与 Chen 等人 (2021) 考虑的环境类似, 不同之处在于钥匙和门不可见, 且奖励是半稀疏的 (钥匙和门均为 $r = 1$)。智能体会被随机重置。回合长度为 50 步。

DMLab 水迷宫: 一个基于经典莫里斯水迷宫 (Morris, 1981) 的部分可观测 3D 视觉 DMLab 环境。任务是导航水迷宫以找到随机生成的活板门。迷宫墙壁有颜色图案, 可用于记住目标位置。观测是大小为 $72 \times 96 \times 3$ 的像素图像。总共有 8 种可能的动作, 包括前进、后退、左移、右移、左转、右转, 以及前进时左转或右转。回合长度为 50, 智能体在地图中心重置。与暗室类似, 智能体无法从观测中看到目标位置, 必须通过奖励推断: 到达则获得 $r = 1$, 否则获得 $r = 0$; 然而, 目标空间是连续的, 因此存在无限多个目标。

4.2 基线

本研究的主要目的是调查 AD 强化学习相对于先前相关工作在多大程度上进行上下文学习。AD 与策略蒸馏 (Policy Distillation) 最为密切相关, 后者通过序列模型从离线交互数据中学习策略。上下文在线元强化学习虽然相关, 但与 AD 不直接可比, 因为 AD 是一种上下文离线元强化学习方法。尽管如此, 我们考虑这两类基线以更好地定位我们的工作。关于这些基线选择的更详细讨论, 请读者参阅附录 B。我们的基线包括:

专家蒸馏 (Expert Distillation, ED): 该基线与 AD 完全相同, 但源数据仅由专家轨迹组成, 而非学习历史。ED 与 Gato (Reed 等, 2022) 最为相似, 区别在于 ED 像 AD 一样建模状态-动作-奖励序列, 而 Gato 建模状态-动作序列。

源算法: 我们将 AD 与基于梯度的源强化学习算法进行比较, 该算法生成用于蒸馏的训练数据。我们将从头运行源算法作为基线, 以比较上下文强化学习与权重内源算法的数据效率。RL² (Duan 等, 2016): 一种在线元强化学习算法, 通过最大化多回合价值函数来联合学习探索和快速上下文适应。RL² 与 AD 不直接可比, 原因类似于在线和离线强化学习算法不直接可比——RL² 在训练期间可以与环境交互, 而 AD 不能。我们使用 RL² 的渐近性能作为 AD 的近似上界。

4.3 评估

预训练后, AD 变换器 P_θ 可以进行上下文强化学习。评估与权重内强化学习算法完全相同, 只是学习完全在上下文内进行, 不更新变换器网络参数。给定一个 MDP (或 POMDP), 变换器与环境交互并填充自己的上下文 (即没有演示), 其中上下文是一个包含最后 c 个转移的队列。然后根据以下指标评估变换器的性能

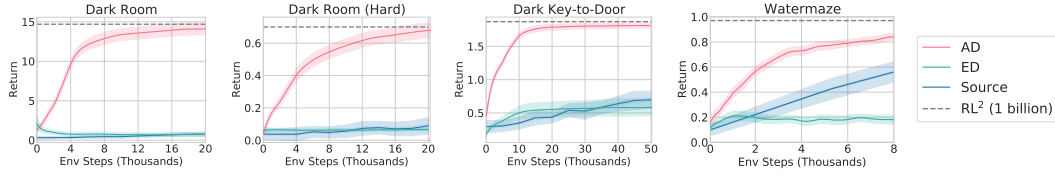


图 4: 主要结果: 我们在需要记忆和探索的环境中评估 AD、 RL^2 、ED 和源算法。在这些环境中, 智能体必须到达只能通过二元奖励推断的目标位置。AD 能够在所有环境中一致地进行上下文强化学习, 并且比其蒸馏的 A3C (“Dark”环境) (Mnih 等, 2016) 或 DQN (Watermaze) (Mnih 等, 2013) 源算法更具数据效率。我们报告了 5 个训练种子 (每个种子 20 个测试种子) 的平均回报 ± 1 标准差。

其最大化回报的能力。对于所有评估运行, 我们在 5 个训练种子上平均结果, 每个训练种子有 20 个评估种子, 总共 100 个种子。任务 $\mathcal{M}$ 从测试任务分布中均匀采样, 并对每个评估种子固定。报告的汇总统计反映了多任务性能。我们分别在 Dark 和 Watermaze 环境中评估 1000 和 160 个回合, 并将性能绘制为测试时总环境步数的函数。

5 实验

这项工作的主要研究问题是, 是否可以通过算法蒸馏将权重内强化学习算法摊销为上下文内算法。上下文内强化学习算法应以与权重内算法相似的方式表现, 并展现出探索、信用分配和泛化能力。我们在一个干净简单的实验设置中开始分析, 该设置需要所有三个属性来解决任务——第 4 节中描述的对抗性赌博机。

为了生成源数据, 我们采样一组训练任务 $\{\mathcal{M}_j\}_{j=1}^N$, 运行上置信界算法 (Lai & Robbins, 1985), 并保存其学习历史。然后我们训练一个变换器来预测动作, 如算法 1 所述。我们评估 AD、ED 和 RL^2 , 并将它们的得分相对于 UCB 和随机策略 $(r - r_{rand.}) / (r_{UCB} - r_{rand.})$ 进行归一化。结果如图 3 所示。我们发现 AD 和 RL^2 都能可靠地上下文内学习从训练分布中采样的任务, 而 ED 不能, 尽管 ED 在分布内评估时确实比随机猜测表现更好。然而, AD 还能上下文内学习解决分布外任务, 而其他方法不能。这个实验表明 AD 可以探索赌博机臂, 一旦到达某个臂就可以通过利用它来分配信用, 并且可以几乎与 UCB 一样好地泛化到分布外。我们现在超越赌博机设置, 在更具挑战性的强化学习环境中研究类似的研究问题, 并将结果作为一系列研究问题的答案呈现。

算法蒸馏是否展现出上下文内强化学习? 为回答这一问题, 我们首先生成用于算法蒸馏的源数据。在暗室与暗钥开门环境中, 我们使用具有 100 个行动者的异步优势演员-评论家算法收集 2000 条学习历史; 在深度思维实验室水迷宫环境中, 我们使用具有 16 个并行行动者的分布式深度 Q 网络收集 4000 条学习历史 (源算法的渐近学习曲线见附录 E, 超参数见附录 M)。如图 4 所示, 算法蒸馏的上下文强化学习在所有环境中均能学习。相比之下, 专家蒸馏在大多数设置下无法进行探索与上下文学习。我们使用训练了 10 亿环境步的强化学习 2 作为元强化学习方法性能上界的代理。尽管算法蒸馏从离线数据中学习强化学习, 其在暗室系列环境上达到了渐近强化学习 2 的水平, 并在水迷宫上接近该水平 (差距在 13% 以内)。

信用分配: 在暗室环境中, 智能体每次访问目标位置时获得 $r = 1$ 。尽管算法蒸馏仅被训练为以单步奖励为条件, 而非以回合回报标记为条件, 它仍能最大化奖励, 这表明算法蒸馏已学会进行信用分配。

探索: 暗室 (困难) 测试智能体的探索能力。由于奖励稀疏 ($r = 1$ 恰好一次), 大部分学习历史中的奖励值为 $r = 0$ 。然而, 算法蒸馏从其上下文中的先前回合推断出目标, 这意味着它已学会探索与利用。

泛化：暗钥匙开门测试在组合任务空间中的分布内泛化。虽然环境总共有 ~ 6.5 千个任务，但训练期间仅见过不到 2 千个。在评估期间，算法蒸馏在大多数未见过的任务上既能泛化又能达到接近最优的性能。

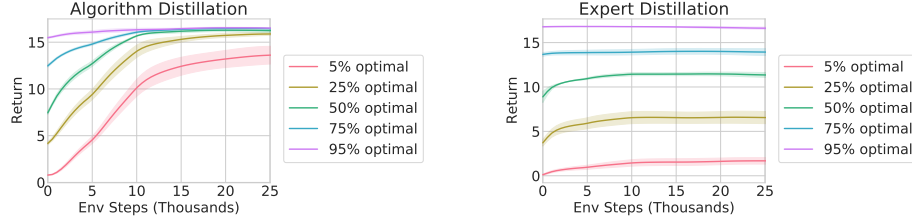


图 5：以部分演示为条件的算法蒸馏与专家蒸馏：我们比较了在暗室（半稠密）上以源算法训练历史中的演示作为提示时，算法蒸馏与专家蒸馏的性能。专家蒸馏仅略有提升并随后保持输入策略的性能，而算法蒸馏能够在上下文中改进该策略，直至策略达到最优或接近最优。

算法蒸馏能否从基于像素的观测中学习？DMLab 水迷宫 是一个基于像素的环境，比暗环境更大，任务从连续均匀分布中采样。该环境在两个方面是部分可观测的——目标在智能体到达之前不可见，第一人称视角限制了智能体的视野。如图 4 所示，算法蒸馏通过上下文强化学习最大化回合回报，而进化蒸馏没有学习。算法蒸馏能否学习到比生成源数据更高效的数据高效的强化学习算法？在图 4 中，算法蒸馏显著比源算法更高效。这一增益是将多流算法蒸馏为单流算法的副产品。源算法（A3C 和 DQN）是分布式的，这意味着它们并行运行多个执行者以获得良好性能。³ A 分布式强化学习算法在总体上可能不是非常数据高效，但每个单独的执行者可以是数据高效的。由于每个执行者的学习历史是单独保存的，算法蒸馏实现了与多流分布式强化学习算法相似的性能，但作为单流方法更数据高效。

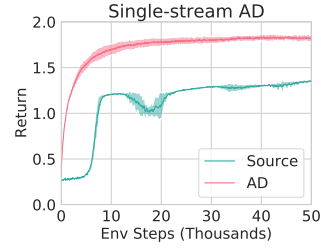


图 6：单流算法蒸馏：算法蒸馏在仅有一个行动者（即单流）的异步优势演员-评论家智能体的学习历史上进行训练。通过对学习历史进行子采样训练（见第 5 节），算法蒸馏学习到了一种数据效率更高的上下文强化学习算法。

这些数据效率增益在蒸馏单流算法时也很明显。在图 6 中，我们展示了通过从单流 A3C 学习历史中每 k 个回合（其中 $k = 10$ ）进行子采样，算法蒸馏仍然可以学习到更高效的数据高效的上下文强化学习算法（更多细节见附录 I）。因此，算法蒸馏可以比多流和单流源强化学习算法都更高效。

虽然算法蒸馏更高效，但源算法实现了略高的渐近性能（见附录 E）。然而，源算法为每个任务 $\mathcal{M}_n$ 生成许多具有唯一权重集 ϕ_n 的单任务智能体，而算法蒸馏生成一个具有固定权重 θ 的通用智能体，该权重在所有任务中保持不变。

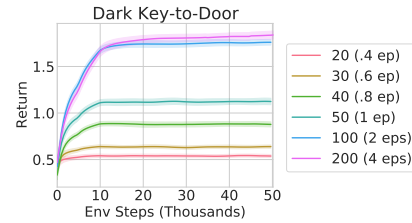


图 7：上下文大小：暗环境钥匙到门中不同上下文大小的算法蒸馏。上下文强化学习仅在上下文大小足够大且跨回合时才会出现。

是否可以通过提示来加速算法蒸馏（AD）演示？ 虽然算法蒸馏可以在不依赖演示的情况下进行强化学习，但它有一个额外的好处，即与源算法不同，它可以以外部数据为条件或通过外部数据进行提示。为了回答研究问题，我们采样

实际上，当前最先进的强化学习算法，如 MuZero (Schrittwieser 等人, 2019) 和 Muesli (Hessel 等人, 2021)，依赖于分布式执行器。

从留出测试集数据中提取策略，这些策略来自源算法历史的不同阶段——从近乎随机的策略到近乎专家的策略。然后，我们用这些策略数据为 AD 和 ED 预填充上下文，并在 Dark Room 环境中运行这两种方法，结果如图 5 所示。ED 保持了输入策略的性能，而 AD 在上下文中不断改进每个策略，直到接近最优。重要的是，输入策略越优，AD 改进它的速度越快，直到达到最优。

上下文强化学习 (RL) 的出现需要多大的上下文规模？ 我们假设 AD 需要足够长的（即跨回合的）上下文才能进行上下文强化学习。我们通过在 Dark Room 环境中训练具有不同上下文规模的多个 AD 变体来检验这一假设。我们在图 7 中绘制了这些不同变体的学习曲线，发现需要 2-4 个回合的跨回合上下文才能学习到接近最优的上下文强化学习算法。当上下文规模大约等于一个回合的长度时，上下文强化学习的初步迹象开始出现。原因可能是上下文足够大，能够保留跨回合信息——例如，在新回合开始时，上下文将填充来自前一个回合大部分时间步的转移。

6 相关工作

离线策略蒸馏：与本文工作最密切相关的是近期利用变换器从离线环境交互数据中学习策略的进展，我们将其称为策略蒸馏 (Policy Distillation, PD)。最初的策略蒸馏架构，如决策变换器 (Decision Transformer, Chen et al., 2021) 和轨迹变换器 (Trajectory Transformer, Janner et al., 2021)，表明变换器能够从离线数据中学习单任务策略。随后，多游戏决策变换器 (Multi-Game Decision Transformer, MGDT, Lee et al., 2022) 和 Gato (Reed et al., 2022) 分别表明策略蒸馏架构也能学习多任务同域和跨域策略。重要的是，这些先前方法使用的上下文长度远小于一个回合的长度，这很可能是这些工作中未观察到上下文强化学习的原因。相反，它们依赖其他方式来适应新任务——多游戏决策变换器微调模型参数，而 Gato 通过专家示范提示来适应下游任务。算法蒸馏 (AD) 无需微调即可在上下文中适应，并且不依赖示范。最后，近期一些工作探索了更通用的策略蒸馏架构 (Furuta et al., 2021)、提示条件化 (Xu et al., 2022) 以及基于在线梯度的强化学习 (Zheng et al., 2022)。

元强化学习：AD 属于学习强化学习的方法类别，也称为元强化学习。具体而言，AD 是一种上下文离线元强化学习方法。这种学习策略改进过程的总体思想在强化学习中有着悠久的历史，但直到最近才局限于元学习超参数 (Ishii 等人, 2002)。Wang 等人 (2016) 和 Duan 等人 (2016) 引入的上下文深度元强化学习方法通常通过环境交互最大化多回合价值函数，并使用基于记忆的架构进行在线训练。在线元强化学习的另一种常见方法包括基于优化的方法，这些方法为元强化学习找到良好的网络参数初始化 (Hochreiter 等人, 2001; Finn 等人, 2017; Nichol 等人, 2018)，并通过额外的梯度步骤进行适应。与其他上下文元强化学习方法一样，AD 是无梯度的——它无需更新网络参数即可适应下游任务。最近的工作提出了从离线数据集中学习强化学习，即离线元强化学习，使用贝叶斯强化学习 (Dorfman 等人, 2021) 和基于优化的元强化学习 (Mitchell 等人, 2021)。鉴于离线元强化学习的困难，Pong 等人 (2022) 提出了一种离线-在线混合策略用于元强化学习。

变换器的上下文学习：在这项工作中，我们区分了上下文学习和增量式或上下文学习。上下文学习涉及从提供的提示或演示中学习，而增量式上下文学习涉及通过试错从自身行为中学习。虽然许多近期工作展示了前者，但展示后者的方法却少得多。可以说，迄今为止最令人印象深刻的上下文学习演示是在文本补全设置中通过提示条件化实现的 (Radford 等人, 2018; Chen 等人, 2020; Brown 等人, 2020)。类似的方法最近被扩展以展示文本条件图像生成中的强大组合性 (Yu 等人, 2022)。近期工作表明，变换器还可以在小规模设置中以上下文方式学习简单的算法类，例如线性回归 (Garg 等人, 2022)。与先前的上下文学习方法一样，Garg 等人 (2022) 需要用专家示例初始化变换器提示。虽然上述方法是上下文学习的例子，但最近的一项工作 (Chen 等人,

2022) 通过将超参数优化视为带有评分函数的序列预测问题, 展示了用于超参数优化的增量式上下文学习。

7 结论

我们证明了算法蒸馏 (Algorithm Distillation) 能够通过使用因果变换器 (Causal Transformer) 对强化学习学习历史进行建模, 将权重内强化学习算法蒸馏为上下文内强化学习算法, 并且算法蒸馏能够学习到比生成源数据更高效的数据利用算法。算法蒸馏的主要局限性在于, 大多数感兴趣的强化学习环境具有较长的回合, 而建模多回合上下文需要比本文所考虑的更强力的长时程序列模型。我们相信这是未来研究的一个有前景的方向, 并希望算法蒸馏能激发研究界对上下文内强化学习的进一步探索。

参考文献

- Charles Beattie, Joel Z. Leibo, Denis Teplyashin, Tom Ward, Marcus Wainwright, Heinrich Küttler, Andrew Lefrancq, Simon Green, Víctor Valdés, Amir Sadik, Julian Schrittwieser, Keith Anderson, Sarah York, Max Cant, Adam Cain, Adrian Bolton, Stephen Gaffney, Helen King, Demis Hassabis, Shane Legg, and Stig Petersen. [DeepMind Lab. CoRR](#), abs/1612.03801, 2016.
- Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. [Language models are few-shot learners](#). *arXiv preprint arXiv:2005.14165*, 2020.
- Stephanie CY Chan, Adam Santoro, Andrew K Lampinen, Jane X Wang, Aaditya Singh, Pierre H Richemond, Jay McClelland, and Felix Hill. [Data Distributional Properties Drive Emergent In-Context Learning in Transformers](#). *arXiv preprint arXiv:2205.05055*, 2022.
- Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Misha Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. [Decision transformer: Reinforcement learning via sequence modeling](#). *Advances in neural information processing systems*, 34:15084–15097, 2021.
- Mark Chen, Alec Radford, Rewon Child, Jeffrey Wu, Heewoo Jun, David Luan, and Ilya Sutskever. [Generative pretraining from pixels](#). In *International Conference on Machine Learning*, pp. 1691–1703. PMLR, 2020.
- Yutian Chen, Xingyou Song, Chansoo Lee, Zi Wang, Qiuyi Zhang, David Dohan, Kazuya Kawakami, Greg Kochanski, Arnaud Doucet, Marc’auelio Ranzato, Sagi Perel, and Nando de Freitas. [Towards Learning Universal Hyperparameter Optimizers with Transformers](#), 2022.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. [Bert: Pre-training of deep bidirectional transformers for language understanding](#). *arXiv preprint arXiv:1810.04805*, 2018.
- Ron Dorfman, Idan Shenfeld, and Aviv Tamar. [Offline Meta Reinforcement Learning—Identifiability Challenges and Effective Data Collection Strategies](#). *Advances in Neural Information Processing Systems*, 34:4607–4618, 2021.
- Yan Duan, John Schulman, Xi Chen, Peter L. Bartlett, Ilya Sutskever, and Pieter Abbeel. [RL²: Fast Reinforcement Learning via Slow Reinforcement Learning](#), 2016.
- Chelsea Finn, Pieter Abbeel, and Sergey Levine. [Model-agnostic meta-learning for fast adaptation of deep networks](#). In *International conference on machine learning*, pp. 1126–1135. PMLR, 2017.
- Hiroki Furuta, Yutaka Matsuo, and Shixiang Shane Gu. [Generalized decision transformer for offline hindsight information matching](#). *arXiv preprint arXiv:2111.10364*, 2021.
- Shivam Garg, Dimitris Tsipras, Percy Liang, and Gregory Valiant. [What Can Transformers Learn In-Context? A Case Study of Simple Function Classes](#), 2022.

- Matteo Hessel, Ivo Danihelka, Fabio Viola, Arthur Guez, Simon Schmitt, Laurent Sifre, Theophane Weber, David Silver, and Hado van Hasselt. [Muesli: Combining Improvements in Policy Optimization](#). In Marina Meila and Tong Zhang (eds.), *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, volume 139 of *Proceedings of Machine Learning Research*, pp. 4214–4226. PMLR, 2021.
- Sepp Hochreiter and Jürgen Schmidhuber. [Long Short-Term Memory](#). *Neural Computation*, 9(8): 1735–1780, 1997.
- Sepp Hochreiter, A Steven Younger, and Peter R Conwell. Learning to learn using gradient descent. In *International conference on artificial neural networks*, pp. 87–94. Springer, 2001.
- Shin Ishii, Wako Yoshida, and Junichiro Yoshimoto. Control of exploitation–exploration meta-parameter in reinforcement learning. *Neural networks*, 15(4-6):665–687, 2002.
- Michael Janner, Qiyang Li, and Sergey Levine. [Reinforcement Learning as One Big Sequence Modeling Problem](#). *arXiv preprint arXiv:2106.02039*, 2021.
- T.L Lai and Herbert Robbins. [Asymptotically Efficient Adaptive Allocation Rules](#). *Adv. Appl. Math.*, 6(1):4–22, mar 1985. ISSN 0196-8858. doi: 10.1016/0196-8858(85)90002-8.
- Kuang-Huei Lee, Ofir Nachum, Mengjiao Yang, Lisa Lee, Daniel Freeman, Winnie Xu, Sergio Guadarrama, Ian Fischer, Eric Jang, Henryk Michalewski, and Igor Mordatch. [Multi-Game Decision Transformers](#), 2022.
- Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. [Offline reinforcement learning: Tutorial, review, and perspectives on open problems](#). *arXiv preprint arXiv:2005.01643*, 2020.
- Eric Mitchell, Rafael Rafailov, Xue Bin Peng, Sergey Levine, and Chelsea Finn. [Offline meta-reinforcement learning with advantage weighting](#). In *International Conference on Machine Learning*, pp. 7780–7791. PMLR, 2021.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. [Playing atari with deep reinforcement learning](#). *arXiv preprint arXiv:1312.5602*, 2013.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. [Human-level control through deep reinforcement learning](#). *nature*, 518(7540):529–533, 2015.
- Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. [Asynchronous Methods for Deep Reinforcement Learning](#). In Maria-Florina Balcan and Kilian Q. Weinberger (eds.), *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*, volume 48 of *JMLR Workshop and Conference Proceedings*, pp. 1928–1937. JMLR.org, 2016.
- Richard G.M. Morris. [Spatial localization does not require the presence of local cues](#). *Learning and Motivation*, 12(2):239–260, 1981. doi: 10.1016/0023-9690(81)90020-5.
- Rafael Müller, Simon Kornblith, and Geoffrey E Hinton. When does label smoothing help? In H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché-Buc, E. Fox, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL <https://proceedings.neurips.cc/paper/2019/file/f1748d6b0fd9d439f71450117eba2725-Paper.pdf>.
- Alex Nichol, Joshua Achiam, and John Schulman. [On First-Order Meta-Learning Algorithms](#), 2018.
- Vitchyr H Pong, Ashvin V Nair, Laura M Smith, Catherine Huang, and Sergey Levine. [Offline meta-reinforcement learning with online self-supervision](#). In *International Conference on Machine Learning*, pp. 17811–17829. PMLR, 2022.
- Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. [Improving language understanding by generative pre-training](#), 2018.

- Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, Eliza Rutherford, Tom Hennigan, Jacob Menick, Albin Cassirer, Richard Powell, George van den Driessche, Lisa Anne Hendricks, Maribeth Rauh, Po-Sen Huang, Amelia Glaese, Johannes Welbl, Sumanth Dathathri, Saffron Huang, Jonathan Uesato, John Mellor, Irina Higgins, Antonia Creswell, Nat McAleese, Amy Wu, Erich Elsen, Siddhant Jayakumar, Elena Buchatskaya, David Budden, Esme Sutherland, Karen Simonyan, Michela Paganini, Laurent Sifre, Lena Martens, Xiang Lorraine Li, Adhiguna Kuncoro, Aida Nematzadeh, Elena Gribovskaya, Domenic Donato, Angeliki Lazaridou, Arthur Mensch, Jean-Baptiste Lespiau, Maria Tsimpoukelli, Nikolai Grigorev, Doug Fritz, Thibault Sottiaux, Mantas Pajarskas, Toby Pohlen, Zhitao Gong, Daniel Toyama, Cyprien de Masson d’Autume, Yujia Li, Tayfun Terzi, Vladimir Mikulik, Igor Babuschkin, Aidan Clark, Diego de Las Casas, Aurelia Guy, Chris Jones, James Bradbury, Matthew Johnson, Blake Hechtman, Laura Weidinger, Iason Gabriel, William Isaac, Ed Lockhart, Simon Osindero, Laura Rimell, Chris Dyer, Oriol Vinyals, Kareem Ayoub, Jeff Stanway, Lorrayne Bennett, Demis Hassabis, Koray Kavukcuoglu, and Geoffrey Irving. [Scaling Language Models: Methods, Analysis & Insights from Training Gopher](#), 2021.
- Scott Reed, Konrad Zolna, Emilio Parisotto, Sergio Gomez Colmenarejo, Alexander Novikov, Gabriel Barth-Maron, Mai Gimenez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, Tom Eccles, Jake Bruce, Ali Razavi, Ashley Edwards, Nicolas Heess, Yutian Chen, Raia Hadsell, Oriol Vinyals, Mahyar Bordbar, and Nando de Freitas. [A Generalist Agent](#), 2022.
- Andrei A. Rusu, Sergio Gomez Colmenarejo, Çağlar Gülçehre, Guillaume Desjardins, James Kirkpatrick, Razvan Pascanu, Volodymyr Mnih, Koray Kavukcuoglu, and Raia Hadsell. Policy distillation. In Yoshua Bengio and Yann LeCun (eds.), *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016. URL <http://arxiv.org/abs/1511.06295>.
- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy P. Lillicrap, and David Silver. [Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model](#). *CoRR*, abs/1911.08265, 2019.
- Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2818–2826, 2016. doi: 10.1109/CVPR.2016.308.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. [Attention is all you need](#). In *Advances in Neural Information Processing Systems*, 2017.
- Jane X Wang, Zeb Kurth-Nelson, Dhruva Tirumala, Hubert Soyer, Joel Z Leibo, Remi Munos, Charles Blundell, Dhharshan Kumaran, and Matt Botvinick. [Learning to reinforcement learn](#), 2016.
- Ronald J. Williams. [Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning](#). *Mach. Learn.*, 8:229–256, 1992. doi: 10.1007/BF00992696.
- Ronald J Williams and David Zipser. [A learning algorithm for continually running fully recurrent neural networks](#). *Neural computation*, 1(2):270–280, 1989.
- Mengdi Xu, Yikang Shen, Shun Zhang, Yuchen Lu, Ding Zhao, Joshua Tenenbaum, and Chuang Gan. [Prompting decision transformer for few-shot policy generalization](#). In *International Conference on Machine Learning*, pp. 24631–24645. PMLR, 2022.
- Jiahui Yu, Yuanzhong Xu, Jing Yu Koh, Thang Luong, Gunjan Baid, Zirui Wang, Vijay Vasudevan, Alexander Ku, Yinfei Yang, Burcu Karagol Ayan, Ben Hutchinson, Wei Han, Zarana Parekh, Xin Li, Han Zhang, Jason Baldridge, and Yonghui Wu. [Scaling Autoregressive Models for Content-Rich Text-to-Image Generation](#), 2022.
- Qinqing Zheng, Amy Zhang, and Aditya Grover. [Online decision transformer](#). *arXiv preprint arXiv:2202.05607*, 2022.

A 环境考量

在本文中，我们考虑零样本泛化困难的环境，因此智能体必须通过试错来学习。我们还希望环境难以对任何特定任务过拟合，以确保我们的方法具有通用性。最后一个实用考量是，我们考虑的环境其跨回合历史能够用因果变换器进行可行建模。鉴于这些考量，我们的评估环境需要满足三个标准：

1. 支持多任务：环境必须是多任务的，以确保我们的智能体和基线不会对任何单一任务过拟合，而是能够在给定领域内的多个任务中进行上下文内强化学习。
2. 任务必须难以推断：为了确保下游任务在零样本情况下难以泛化，我们使用需要探索的环境。即，我们要求环境要么任务只能从奖励中推断而无法从观察中推断，要么任务是部分可观测的。
3. 支持多回合上下文：最后，我们施加一个实用约束——环境回合必须足够短，以便普通的类 GPT 变换器能够在其上下文中容纳多个回合。由于本文引入算法蒸馏作为一种方法，我们希望在使用规范架构的最干净设置中研究它。我们将使用能够扩展到更长序列的更复杂架构来研究算法蒸馏留作未来工作。

先前相关工作（Chen 等人，2021；Lee 等人，2022；Janner 等人，2021；Reed 等人，2022）在 Atari、OpenAI Gym 以及其他环境中进行了评估。然而，Atari 和 OpenAI Gym 并不满足上述至少一项标准。Atari 和 OpenAI Gym 的回合通常较长，每个回合可能包含数千次或更多的转移，因此在因果变换器的上下文中填充跨回合历史在技术上具有挑战性。实际上，先前相关工作仅考虑了回合内的上下文长度。此外，在 Atari 和 OpenAI Gym 中，通常很容易仅从观测或密集奖励中推断出任务，这降低了对探索的需求。基于这些原因，我们改为在满足全部三项标准的环境中进行评估。

B 密切相关的先前方法

在主要结果中，我们使用专家蒸馏（Expert Distillation，ED）作为基线。在此，我们讨论最密切相关的方法与 AD 有何不同，以及为何 ED 足以支持本文的主张。

专家蒸馏（ED）：ED 与 Gato（Reed 等人，2022）最为相似，后者使用因果变换器对来自收敛强化学习策略的专家序列进行建模。ED 同样训练一个因果变换器，利用专家策略数据来预测动作。ED 与 Gato 之间存在两个关键差异。首先，与 Gato 使用（相对于回合长度而言）较小的回合内上下文不同，ED 在与 AD 相同的跨回合上下文上进行训练，因此 ED 和 AD 所使用的架构是相同的。因此，AD 的优势不能仅归因于跨回合上下文，还应归因于用于训练 AD 的离线数据中的学习进展。其次，ED 建模状态-动作-奖励序列，而 Gato 仅建模状态-动作序列。ED 与 AD 之间的主要区别在于，AD 是在完整的多任务学习历史上进行训练，而非专家策略数据。

决策 Transformer（Decision Transformer，DT）（Chen 等人，2021）与多游戏决策 Transformer（Multi-Game Decision Transformer）（Lee 等人，2022）：决策 Transformer 从强化学习智能体收集的单任务离线数据中学习以回报为条件的策略。尽管训练数据本身（强化学习智能体的经验回放缓冲区）包含学习过程，但决策 Transformer 所使用的上下文长度过小，无法捕捉任何学习进展，也无法利用跨回合信息来识别任务。例如，Atari 实验使用的上下文长度为 30 – 50 个标记，即 10 – 17 次转移。Atari 游戏单个回合可能包含数百或数千次转移，这意味着这些上下文主要捕捉的是回合内信息。此外，在如此多的转移范围内，底层经验回放缓冲区数据中几乎不会发生学习进展。

DT 与 AD / ED 的另一个区别在于，DT 学习以回报为条件的模型，而 AD / ED 均以奖励为条件。在我们的设定中，仅以回报为条件无法产生最优策略，因为智能体在探索环境并利用跨回合上下文识别任务之前并不知道任务是什么。由于 (i) DT 使用较小的回合内上下文，且 (ii) 在所考虑的环境中回报条件化无济于事，因此该基线与 ED 类似

但仅使用较小的回合内上下文，这严格弱于我们所考虑的 ED 的长跨回合上下文变体。

轨迹变换器 (Trajectory Transformer, TTO) (Janner 等, 2021)：与 AD 类似，TTO 也对状态-动作-奖励标记进行建模，但除了预测动作外，它还通过预测状态和奖励来学习世界模型。为了最大化回报，TTO 随后使用束搜索选择高奖励动作。然而，在我们的设定中，TTO 会遇到与 DT 相同的问题。为了准确建模奖励，它需要更长的跨回合上下文，因为一个环境支持多个任务。与 DT、MGDT 和 Gato 类似，TTO 使用较小的回合内上下文。因此，在我们考虑的设定中，TTO 的表现不会优于 DT、MGDT 或 ED。我们还注意到，与 TTO 相比，AD 是无模型的。在 AD 中，动作从变换器基于历史的条件预测中采样，回报最大化源于对强化学习算法学习历史的建模。

总而言之，AD 与先前方法的主要区别在于其上下文是跨回合的，因此足够大，能够捕捉学习进展和任务信息。AD 还可以通过学习世界模型（如 TTO）或像 DT 那样以回报为条件来进一步增强，但这些研究更适合作为未来工作，因为它们与本文所解决的主要研究问题——通过模仿具有长跨回合上下文的强化学习算法的学习历史，上下文强化学习能否涌现——是相切的。

AD 也与上下文元强化学习的先前工作密切相关。虽然 AD 和上下文元强化学习都使用基于记忆的架构对跨回合历史进行建模，但先前的上下文元强化学习算法（如 RL2 (Duan 等人, 2016)）是在线训练的，并依赖于使用 TD 学习来学习多回合价值函数，而 AD 是离线训练的，并使用监督模仿学习目标。

C 专家蒸馏主要结果

我们进一步阐述图 4 中的主要结果，并提供关于 ED 基线行为的直觉。在 Dark Room、Dark Room（困难）和 Watermaze 中，ED 的性能要么持平，要么下降。原因是 ED 在训练期间只看到专家轨迹，但在评估期间，它需要首先探索（即执行非专家行为）来识别任务。这种所需的行为对 ED 来说是分布外的，因此它不会进行上下文强化学习。在 Dark Key-to-Door 中，智能体在每个回合开始时被随机重置，而在所有环境中，智能体的起始位置是固定的。由于随机重置，ED 智能体有时会被重置到 Dark Key-to-Door 中的第一个目标附近，这使它偶尔能够识别任务的第一个目标，这就是它表现出轻微改进的原因。

D 模型规模

我们在图 8 中研究了变换器容量如何影响性能。虽然在所有研究的模型规模下都涌现了上下文强化学习，但我们发现增加模型深度、以嵌入维度衡量的模型宽度，以及（在较小程度上）注意力头的数量，可以提高 Dark Key-to-Door 上的性能。

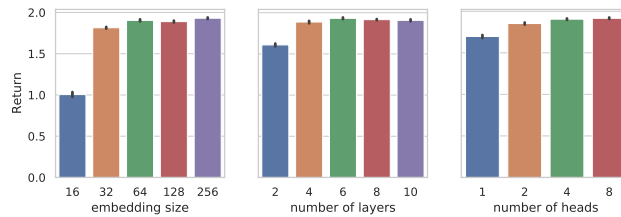


图 8：模型规模研究：我们研究了增加模型容量如何影响 AD。虽然 AD 的上下文强化学习无论模型容量如何都会涌现，但增加模型深度和宽度有助于改进 AD，直到它达到接近最优的性能。

E 源算法训练运行

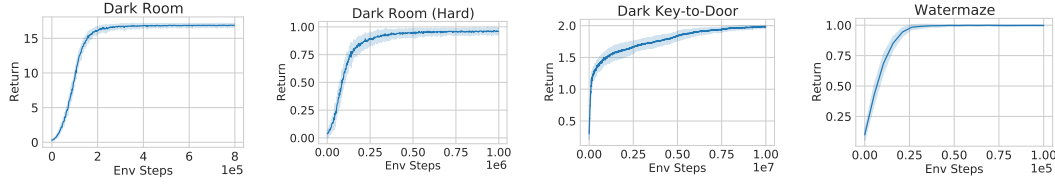


图 9: A3C (Mnih 等人, 2016) 和 DQN (Mnih 等人, 2013) 的 $Q-\lambda$ 变体在 Dark 和 Watermaze 环境中用于生成学习历史的渐近性能。这些曲线展示了 AD 训练所用的学习历史。图 4 中绘制的源算法与这些图中的相同。

F 标签平滑消融

对于更困难的探索任务 Dark Room (Hard)，我们发现添加标签平滑正则化 (Szegedy 等人, 2016; Müller 等人, 2019) 提高了 AD 的上下文学习能力。在图 10 中，我们消融了使用标签平滑对 3 个不同 α 值以及关闭标签平滑时的益处。图中的每条曲线表示 5 个训练种子的平均性能。我们可以看到，在一定范围内添加标签平滑可以提高算法蒸馏的上下文学习能力，性能随着评估回合数的增加而持续提升。

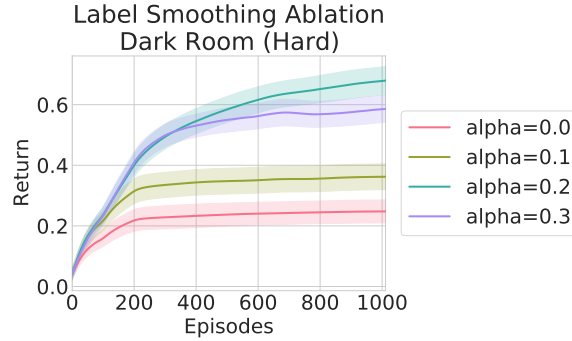


图 10: 在 Dark Room (Hard) 上使用不同标签平滑量训练的 AD。

G RL^2 网络架构: 变换器与 LSTM

我们比较了使用变换器作为 RL^2 的架构而不是 LSTM。在图 11 中，我们在 Dark Room 环境中运行了变换器和 LSTM RL^2 智能体。所示曲线是在学习率和展开长度超参数扫描中的最佳结果。变换器架构为 4 层，模型大小为 256，采用前置层归一化。虽然两种智能体达到了相似的最终性能水平，但所有训练的 RL^2 变换器模型往往更不稳定，平均回报不如 LSTM 架构一致。鉴于基于变换器的 RL^2 在更简单的 Dark Room 设置上表现不佳，我们的其他实验设置使用了 LSTM。

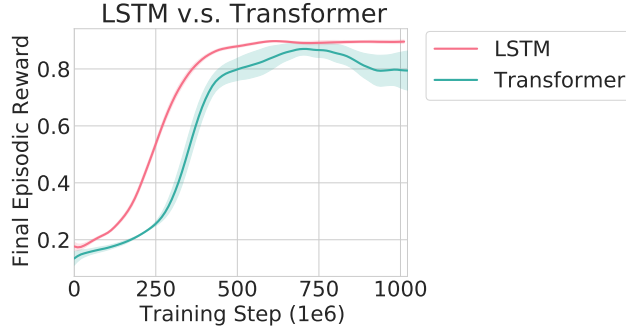


图 11: 在 Dark Room 上 RL 2 智能体的 LSTM 与变换器架构比较。每条曲线在 5 个训练种子上取平均, 阴影区域表示标准误。

H 源数据中的训练任务数量

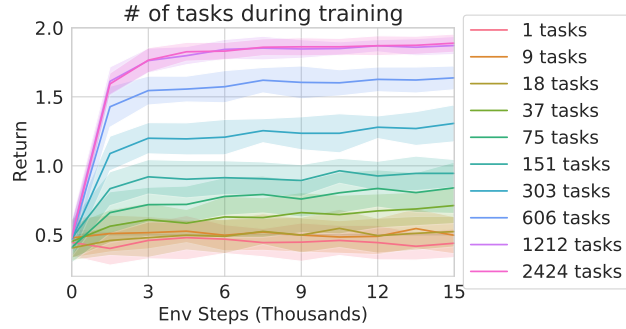


图 12: 在暗钥匙开门任务上, 使用不同数量训练任务进行算法蒸馏训练, 并在固定测试任务集上评估 300 个回合。

一个有趣的问题是, 算法蒸馏需要训练多少个任务才能学会一种能泛化到未见过任务的算法。我们在不同数量的暗钥匙开门训练任务上训练算法蒸馏, 并在同一组测试任务上评估所得模型。图 12 展示了所得算法蒸馏模型在测试任务集上的上下文学习曲线。提醒一下, 暗钥匙开门任务共有 $81^2 = 6561$ 个独特任务。在 1 个、9 个或 18 个训练任务上训练的模型在测试任务上未表现出任何上下文学习。而在 37 个、75 个和 151 个任务上训练的模型虽然整体性能不佳, 但在 300 个回合过程中确实表现出一定的上下文学习。最佳模型是在 1212 个和 2424 个任务上训练的, 这分别约占暗钥匙开门领域任务总数的 18% 和 37%。

I 单流算法蒸馏

我们提供更多关于图 6 所示单流结果实验设置的细节。我们在图 4 中展示, 当算法蒸馏在分布式源强化学习算法的部分智能体数据上训练时, 所得模型比源算法更具数据效率。这里我们确认算法蒸馏在单流设置下能产生比其训练所用算法更快的算法。本实验中, 我们在 2048 个暗钥匙开门任务上训练 A3C, 每个任务 2000 个回合。然后我们在所得数据上训练算法蒸馏, 同时将学习历史按 10 倍因子进行子采样。更具体地说, 我们从每个学习历史中取每第 10 个回合, 从而为每个任务生成 200 个回合的压缩学习轨迹。图 6 将所得算法蒸馏模型在测试任务集上的评估结果与源算法在这些任务上的性能进行比较。算法蒸馏学到的模型学习速度远快于源算法, 证实了算法蒸馏能将缓慢的基于梯度的算法转变为数据效率高得多的上下文学习算法。

J 随机掩码

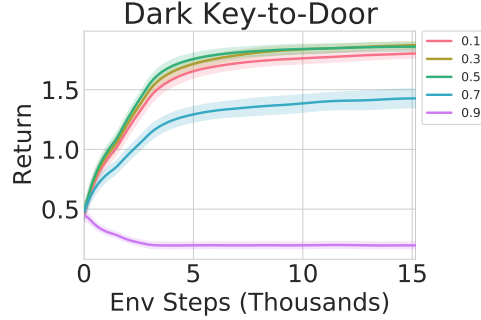


图 13: 在 9x9 单目标网格世界中，训练期间不同随机掩码值下算法蒸馏的下游性能。

在训练过程中，输入令牌被随机掩码以避免对训练数据过拟合。该图展示了在 9x9 暗黑钥匙门（Dark Key-to-Door）领域中，不同随机掩码值对下游结果的影响。 $0.3 - 0.5$ 的值表现最佳，所有实验均选用 0.3 的值。

K AD 网络架构：变换器与长短期记忆网络对比

本文通过将算法蒸馏（AD）与基于长短期记忆网络（LSTM, Hochreiter & Schmidhuber, 1997）的 AD 进行比较，考察变换器（Transformer）架构对算法蒸馏成功的重要性。具体而言，LSTM 接收截至最近时间步 $t-1$ 的 (o_i, a_i, r_i) 三元组的拼接嵌入。随后，LSTM 的输出与当前观测 o_t 的嵌入拼接，二者共同输入多层感知机（MLP）策略躯干，以生成当前动作 a_t 的分布。LSTM 隐藏层大小（512）、MLP 深度（2）和 MLP 宽度（256）通过网格搜索基于下游奖励获取进行扫描和调优。

在暗黑钥匙门任务上比较变换器 AD 与 LSTM AD，本文发现两种智能体均具备上下文学习能力，表明 AD 的成功并不依赖于底层网络架构。然而，本文还发现变换器变体始终优于 LSTM 变体，因此本文其余实验均采用变换器变体。这一发现与近期变换器架构在序列预测任务中相对于循环神经网络（RNN）架构的更广泛成功相一致。

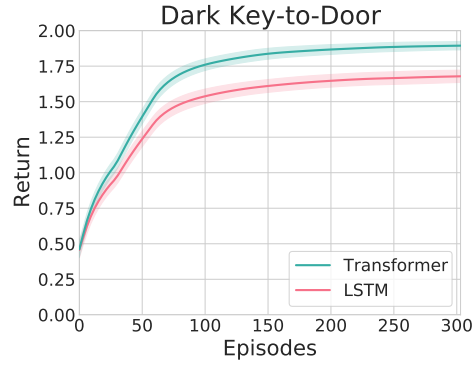


图 14: 暗黑钥匙门上变换器与 LSTM 架构的算法蒸馏对比。均值 ± 1 个标准差，基于 5 个训练种子和 20 个评估种子。300 个回合对应 15k 环境步数。

L 算法蒸馏超参数

Hyperparameter	Dark Room	Dark Room (Hard)	Dark Key-to-Door	Watermaze
Embedding Dim.			64	
Number of Layers			4	
Number of Heads			4	
Feedforward Dim.			2048	
Position Encodings			Absolute	
Layer Norm Placement			Post Norm	
Dropout Rate			0.1	
Attention Dropout Rate	0.5	0	0.5	0.5
Sequence Mask Prob	0.3	0.5	0.3	0.3
Label Smoothing α	0	0.2	0	0

表 1: 算法蒸馏架构超参数。

Hyperparameter	Value
Batch Size	128
Optimizer	Adam
β_1	0.9
β_2	0.99
Gradient Clip Norm Threshold	1
Learning Rate Schedule	Cosine Decay
Initial Value	2e-6
Peak Value	3e-4

表 2: 算法蒸馏优化超参数。

Layer	Hyperparameter	Value
<i>Conv Block</i>		
Conv	Channel	128
	Kernel	5
	Stride	2
BatchNorm	Decay Rate	0.999
	eps	1e-5
Activation	-	ReLU
Max Pooling	Kernel	2
	Stride	2
Dropout	Rate	0.2
<i>Network</i>		
Conv Blocks	-	3
Final Linear Layer	Units	256

表 3: 水迷宫图像编码器超参数。

M 源强化学习算法超参数

M.1 黑暗环境

Hyperparameter	Value
Batch Size (Num. Actors)	100
λ	0.95
Agent Discount	0.99
Entropy Bonus Weight	0.01
MLP Layers	3
MLP Hidden Dim	128
Optimizer	Adam
β_1	0.9
β_2	0.999
ϵ	1e-6
Learning Rate	1e-4

表 4: 黑暗环境下源 A3C 算法超参数。

M.2 DMLAB 水迷宫

Hyperparameter	Value
Batch Size	8
Rollout Length	40
Rollout Overlap	31
Number of Actors	16
Reply Buffer Capacity	1e5
Offline Data Fraction	0.7
λ	0.75
ϵ	0.01
Agent Discount	0.9
Target Update Period	50
ResNet Channels	[32, 64, 64]
ResNet Kernels	[3, 3, 3]
ResNet Strides	[1, 1, 1]
Pool Kernels	[3, 3, 3]
Pool Strides	[2, 2, 2]
Optimizer	Adam
β_1	0.9
β_2	0.999
ϵ	1e-6
Gradient Clip Norm Threshold	10
Learning Rate	1e-4

表 5: 水迷宫源 DQN($Q-\lambda$) 算法超参数。N RL² H 超参数

Hyperparameter	Value
RL Algorithm	A3C
Learning Rate	3e-4
Batch Size	256
Unroll Length	20
LSTM Hidden Dim.	256
LSTM Number of Layers	2
Episodes Per Trial	10

表 6: “黑暗”环境中使用的强化学习² 超参数。

Hyperparameter	Value
RL Algorithm	DQN($Q-\lambda$)
Learning Rate	1e-4
Batch Size	96
Unroll Length	40
LSTM Hidden Dim.	256
LSTM Number of Layers	1
Episodes Per Trial	30

表 7: 水迷宫环境中使用的 RL² 超参数。

O 注意力图

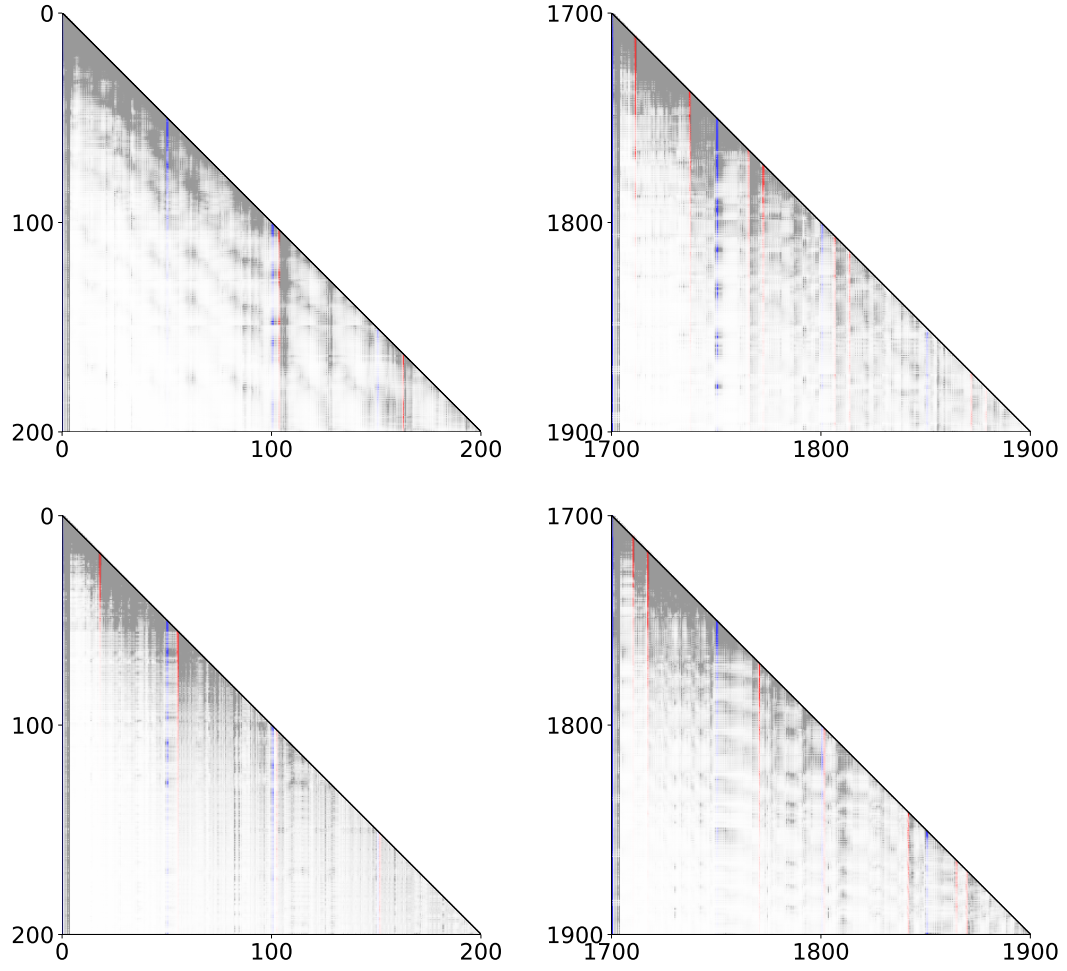


图 15: 来自五个独立种子的 AD 注意力图。白色和灰色分别对应低注意力和高注意力。红色和蓝色表示这些转移分别对应回合重启和正奖励标记。左列绘制了评估 200 个时间步后（上下文初始填充时）AD 变换器的注意力。右列绘制了评估 1900 步（38 个回合）后的注意力。每个回合长度为 50 步。从这些模式中可以看出，AD 会关注跨越多个回合的标记来预测其下一个动作。